

Science-Bitch: Complete Command Reference & Guide

Summary: Science-Bitch is a comprehensive command-line tool that implements the complete scientific method workflow for systematic hypothesis testing, controlled experimentation, and...

Generated: 2026-01-15 03:57 UTC | Ideas Extracted: 56

What Happened

Science-Bitch is a comprehensive command-line tool that implements the complete scientific method workflow for systematic hypothesis testing, controlled experimentation, and evidence-based conclusions.

Don't use `/science-bitch` when: ❌ Just need quick testing (use direct testing) ❌ Don't need systematic approach ❌ Hypothesis is already verified ❌ Just need data collection (use data tools directly) ❌ Time-constrained (full workflow takes time).

Key Benefits: Systematic Approach: Structured workflow ensures completeness Evidence-Based: All conclusions traceable to data Reproducible: State capture enables reproduction Professional: High-quality documentation Integrated: Works with WAFT ecosystem.

This command provides: Complete Scientific Method: Full workflow from hypothesis to conclusion State Capture: Initial state (A) and final state (B) tracking Data Collection: Systematic data collection during experiments (C) Hypothesis Testing: Form, test, and verify hypotheses Report Generation: Generate field guides and project status reports Evidence-Based Analysis: Systematic analysis with traceable evidence.

Complete Scientific Method Implementation: Full workflow from hypothesis formation to conclusion State Capture System: Automatic tracking of system states before (A) and after (B) experiments Systematic Data Collection: Comprehensive data collection during experiments (C) Hypothesis Testing Framework: Structured approach to forming, testing, and verifying hypotheses Automated Report Generation: Professional PDF reports with full documentation Evidence-Based Analysis: Systematic analysis with traceable evidence chains.

Process: Identify the question or phenomenon to investigate Formulate a clear, testable hypothesis Define variables (independent, dependent, control) Specify what would verify or refute the hypothesis.

Process: Define experiment structure Specify variables and their types Design data collection methods Plan state capture points (A and B) Define success criteria.

Use when: Hypothesis and design already exist Want to re-run experiment Need to skip interactive prompts.

This command integrates with: Scientific Method Tool: `scientific_method_tool/` provides core functionality PDF Generation: Uses `waft.evolution.pdf_generator` for reports Work Efforts: Tracks progress in `WE-260112-az3z` State Capture: Captures system states before/after experiments Data Collection: Records all measurements during experiments.

Core functionality provided by `scientific_method_tool/` : Hypothesis formation and validation Experiment design State capture utilities Data collection framework Analysis tools.

Solution: Ensure `scientific_method_tool` is available Check:

```
python3 -c "from scientific_method_tool import Hypothesis" .
```

Science-Bitch provides a complete, systematic approach to scientific research and experimentation. By implementing the full scientific method workflow with automatic state capture, systematic data collection, and professional report generation, it enables rigorous, evidence-based research with full traceability.

Key Components: Independent Variables: What you're changing Dependent Variables: What you're measuring Control Variables: What stays constant Data Collection Plan: When and how to collect data State Capture Points: When to capture system state.

Key Methods: `run_interactive()` : Run full interactive workflow `generate_field_guide()` : Generate field guide PDF `generate_project_status_report()` : Generate status report PDF `_create_field_guide_content()` : Create field guide markdown `_create_project_status_content()` : Create status report markdown.

[Overview](#) [Quick Start](#) [Complete Workflow Sequence](#) [Command Options](#) [Usage Examples](#) [Scientific Method Cycle](#) [File Structure](#) [Integration](#) [Best Practices](#) [Troubleshooting](#) [Technical Implementation](#) [Related Commands](#).

Components Captured: System configuration Current metrics Environment state Resource usage Any relevant system state.

```
{  "state_id":  "state_a",  "timestamp":  "2026-01-14T19:44:33Z",  "state_hash":  "abc123...",  "components": { "fitness_score": 0.65, "iteration_count": 0, "system_load": 0.3 } }
```

Process: Execute experiment according to design Collect data points during execution (C) Record measurements and observations Track experiment progress Handle errors and edge cases.

Process: Capture final state snapshot Generate state hash for comparison Compare with initial state (A) Identify changes Save state to `_science/experiments/[exp_id]/state_b.json`.

```
{  "state_id":  "state_b",  "timestamp":  "2026-01-14T19:50:00Z",  "state_hash":  "def456...",  "components": { "fitness_score": 0.85, "iteration_count": 15, "system_load": 0.5 },  "changes_from_a": { "fitness_score": { "old": 0.65, "new": 0.85, "delta": 0.20 }, "iteration_count": { "old": 0, "new": 15, "delta": 15 }, "system_load": { "old": 0.3, "new": 0.5, "delta": 0.2 } } }
```

Analysis Performed: Compare states A and B Analyze collected data (C) Verify or refute hypothesis Calculate confidence level Generate conclusions Identify patterns and insights.

What it does: Prompts for hypothesis formation Guides through experiment design Captures initial state
(A) Runs experiment with data collection (C) Captures final state (B) Analyzes results Generates report.

What it does: Starts with pre-defined hypothesis Skips hypothesis formation phase Proceeds directly to experiment design Runs complete workflow.

What it does: Prompts for hypothesis formation Guides through experiment design Captures initial state
(A) Runs experiment with data collection (C) Captures final state (B) Analyzes results Generates report.

What it does: Generates comprehensive field guide PDF Documents all workflow phases Provides examples and best practices Includes command reference.

What it does: Generates project status report PDF Shows current state and goals Documents evidence collected Lists next steps.

What it does: Starts with pre-defined hypothesis Skips hypothesis formation phase Proceeds directly to experiment design Runs complete workflow.

Observe: Identify phenomenon or question Hypothesize: Form testable hypothesis Design: Create experiment with variables Capture State A: Initial system state Run Experiment: Execute with data collection Collect Data C: Measurements during experiment Capture State B: Final system state Analyze: Verify/refute hypothesis Report: Generate conclusions and documentation Iterate: Modify variables and repeat.

Additional Details

science/ | — README.md # Overview and quick start | — SUMMARY.md # Implementation summary
| — experiments/ # Experiment definitions and results | | — [exp_id]/ # Individual experiment | |
| — experiment.json # Experiment definition | | | — state_a.json # Initial state (A) | | | —
state_b.json # Final state (B) | | | — results.json # Experiment results | — data/ # Collected data (C)
| | — [exp_id]/ # Data for each experiment | | — data_series.json # Collected measurements | —
reports/ # Generated reports | | — field_guide.pdf # Usage guide | | — field_guide.md # Source
markdown | | — project_status.pdf # Status report | | — project_status.md # Source markdown |
| — experiment*.md # Experiment reports | — tools/ # Helper utilities | — gener-
ate_field_guide_pdf.py | — generate_status_pdf.py.

Tracks progress in work effort: WE-260112-az3z_s-
cience_bitch_command_full_scientific_method_cli Tickets and progress tracking Implementa-
tion documentation.

Clear Hypotheses: Formulate specific, testable hypotheses Be specific about what you're testing Define
clear success criteria Specify variables explicitly.

Systematic Design: Plan experiments carefully Control for confounding variables Design reproducible
experiments Plan data collection points.

Complete State Capture: Capture all relevant components Identify all system components Capture
comprehensive state Generate state hashes for comparison.

Careful Analysis: Verify all claims with evidence Compare states systematically Analyze data trends
Calculate confidence levels.

Document Everything: Generate reports for future reference Create comprehensive reports Include all
evidence Document conclusions.

Iterate: Refine hypotheses and experiments based on results Learn from each experiment Refine hypotheses Improve experimental design.

Solution: Ensure you're in a project with `waft` installed Check: `waft --help` should show `science-bitch` command.

When `/science-bitch` is executed: AI reads this command file Executes `waft science-bitch` with appropriate options Processes results and displays output Generates reports if requested.

`/hypothesis` : Form and verify hypotheses (complementary workflow) `/prove-it` : Prove scientific method tool works `/verify` : Verify claims with evidence `/analyze` : Analyze data and generate insights `/checkpoint` : Document current state `/reflect` : Capture learnings and insights.

Use `/science-bitch` when: ☒ Need to test a hypothesis systematically ☒ Want to run controlled experiments ☒ Need evidence-based conclusions ☒ Want to track state changes ($A \rightarrow B$) ☒ Need systematic data collection ☒ Want to generate scientific reports ☒ Need to verify or refute a hypothesis.

After completion, provides: Hypothesis: Testable hypothesis statement Experiment Design: Complete experiment structure State Snapshots: Initial (A) and final (B) states Collected Data: All measurements (C) Analysis: Hypothesis verification/refutation with confidence Report: Comprehensive documentation (if requested).

Resources: Work Effort: `WE-260112-az3z_science_bitch_command_full_scientific_method_cli`
Source Code: `src/waft/core/science_bitch.py` Documentation: `_science/README.md` Examples: `scientific_method_tool/example_usage.py` .

This guide provides complete documentation for the Science-Bitch command, covering all aspects from quick start through advanced usage, troubleshooting, and technical implementation.

Runs the complete interactive scientific method workflow.

Purpose: Create experiment design based on hypothesis.

Output: Experiment design with variables, data collection plan, and state capture strategy.

Purpose: Systematic data collection during experiment.

Output: Final state snapshot with comparison to initial state.

Current project state Goals and objectives Evidence collected Next steps Progress tracking.

Generates comprehensive field guide PDF showing how to use the command.

Content: Current project state Goals and objectives Evidence collected Next steps Progress tracking.

Starts workflow with a hypothesis already defined.

Runs experiment without interactive prompts (uses existing hypothesis/design).

Output: Complete experiment with analysis and report.

The command implements the complete scientific method:.

Solution: Check file permissions on `_science/` directory Fix: `chmod -R u+w _science/` .